Kalman Filter Milestone-2
Full-Body 3D Gait Estimation

Ali Hamza(30618), Huzaifa Iqbal(31563), Usman Hussain(28992), Sufyan Sohail(31567)

March 15, 2026

Contents

1 Overview and Objectives **3**

2 Dataset Description **3**

3 State Vector Definition **3**
3.1 Single Joint State Vector . . . 3
3.2 Full Body State Vector . . . 3

4 Linear Kalman Filter (LKF) **3**
4.1 Mathematical Model . . . 3
4.2 Software Architecture . . . 4
4.2.1 Separation of concerns. . . . 4
4.2.2 Static linkage of filter functions. . . . 4
4.2.3 The `LKFState` struct. . . . 4
4.3 Prediction Step . . . 4
4.4 Update Step . . . 5
4.4.1 Innovation residual. . . . 5
4.4.2 Pre-caching PH^T. . . . 5
4.4.3 Cholesky solve and innovation covariance symmetrisation. . . . 5
4.5 Implementation Details: Joseph Form Covariance Update . . . 5
4.6 Matrix Construction and Initialisation . . . 6
4.6.1 Noise parameters. . . . 6
4.6.2 Cold-start initialisation. . . . 6
4.7 Main Filter Loop . . . 6
4.8 CSV Output Format . . . 7
4.9 Command-Line Interface and Parameter Tuning . . . 7

5 Extended Kalman Filter **7**
5.1 Why an Extended Kalman Filter? . . . 7
5.2 EKF Algorithm . . . 8
5.2.1 Predict . . . 8
5.2.2 Update . . . 8
5.3 EKF Implementation . . . 8
5.3.1 `EKFState` — Data Structure . . . 8
5.3.2 `wrap_angle` and `wrap_innovation` — Angle Utilities . . . 8
5.3.3 `make_R_adaptive` — Per Step Noise Matrix . . . 9
5.3.4 `cartesian_to_spherical` — Measurement Conversion . . . 9
5.4 `ekf_predict` — Prediction Step . . . 10
5.5 `ekf_update` - Update Step . . . 10
5.6 `run_ekf` — Main Pipeline . . . 11

6 Matrix Inversion Strategy: Cholesky Decomposition **11**
6.1 Implementation Details . . . 11
6.2 Architectural and Numerical Justification . . . 12

7 Memory Management Strategy: Dynamic Heap Allocation **13**
7.1 Implementation Details . . . 13
7.2 Architectural Justification . . . 13

8 Manual Math Approximations **14**
8.1 Arctangent Approximation: The CORDIC Algorithm . . . 14
8.2 Square Root Approximation: Newton Raphson Method . . . 15

9 Results, Analysis, and Performance Evaluation **15**
9.1 Implementation Parameters . . . 15
9.1.1 Parameter Justification . . . 15
9.2 Computational Performance . . . 16
9.3 Quantitative Performance: RMSE Analysis . . . 16
9.4 Position Estimation . . . 17
9.5 Velocity, Acceleration, and Jerk Estimation . . . 18
9.6 Full State Overview . . . 20
9.7 LKF vs EKF Direct Comparison . . . 21
9.8 Discussion . . . 22

10 Deliverables **23**

1 Overview and Objectives

In this report, we transition from mathematical modeling to a full scale systems implementation. Building upon the models derived in milestone 1, in this project we implemented both a Linear Kalman Filter (LKF) and an Extended Kalman Filter (EKF) in c++. The implementation is applied to a high dimensional full-body 3D human gait dataset, requiring explicit management of memory, numerical stability, and computational efficiency.

2 Dataset Description

The provided dataset contains full body 3D human gait motion data captured across 23 distinct joints. Each joint record consists of 3D position measurements (x, y, z). Both the LKF and EKF algorithms were implemented to process and filter this specific high dimensional dataset.

3 State Vector Definition

To accurately model the kinematics of the human gait, the system tracks position (p), velocity (v), acceleration (a), and jerk (j) for each spatial axis.

3.1 Single Joint State Vector

For an individual joint, the state vector is a 12×1 column vector encompassing the kinematic states across the X, Y, and Z axes. It is defined as:

$$X_{joint} = \begin{bmatrix} p_x \\ v_x \\ a_x \\ j_x \\ p_y \\ v_y \\ a_y \\ j_y \\ p_z \\ v_z \\ a_z \\ j_z \end{bmatrix} \tag{1}$$

3.2 Full Body State Vector

To track the entire body simultaneously, a single, large state vector is constructed by stacking the individual state vectors of all 23 joints. This results in a highly dimensional 276×1 column vector (23 joints $\times$ 12 states/joint):

$$X_{full} = \begin{bmatrix} X_{joint_1}^T & X_{joint_2}^T & \dots & X_{joint_{23}}^T \end{bmatrix}^T \tag{2}$$

This massive state space introduces significant computational challenges, necessitating the highly optimized C++ matrix operations detailed in subsequent sections.

4 Linear Kalman Filter (LKF)

4.1 Mathematical Model

The LKF operates on linearised Cartesian joint position measurements. Because both the motion model and the sensor model are linear, no linearisation approximation is required, the filter is exact under Gaussian noise

assumptions. The prediction step projects the state estimate and its uncertainty forward by one timestep Δt:

$$\hat{x}_{n|n-1} = F\,\hat{x}_{n-1|n-1} \tag{3}$$

$$P_{n|n-1} = F\,P_{n-1|n-1}\,F^T + Q \tag{4}$$

For the update step, the Kalman gain K_n is calculated by weighting the filter's own uncertainty against the measurement noise, and the state estimate is corrected by the scaled innovation residual y_n:

$$K_n = P_{n|n-1}\,H^T\big(H\,P_{n|n-1}\,H^T + R\big)^{-1} \tag{5}$$

$$\hat{x}_{n|n} = \hat{x}_{n|n-1} + K_n\big(z_n - H\,\hat{x}_{n|n-1}\big) \tag{6}$$

H acts as a selector that extracts only the position rows from the full state and discards velocity, acceleration, and jerk.

4.2 Software Architecture

4.2.1 Separation of concerns.

The implementation splits responsibilities across three headers: `matrix_utils.h` (dense matrix arithmetic), `csv_reader.h` (data ingestion and the `GaitDataset` abstraction), and `kalman_matrices.h` (construction of F, Q, H, R, x_0, and P_0). Keeping the matrix construction logic in `kalman_matrices.h` means that tuning the physical model, changing the noise densities or the kinematic order, it requires no edits to `lkf.cpp` itself, reducing the risk of accidentally breaking the filter loop.

4.2.2 Static linkage of filter functions.

Both `lkf_predict` and `lkf_update` are declared `static`, which gives them internal linkage. This is an encapsulation decision: the functions are implementation details of this translation unit and should never be called from external code. Marking them `static` allows the compiler to inline them aggressively and also prevents symbol name clashes.

4.2.3 The `LKFState` struct.

Rather than passing $\hat{x}$ and P as separate arguments and return values, they are bundled into a struct:

```
struct LKFState {
    Matrix x;   // state estimate (276x1)
    Matrix P;   // covariance (276x276)
};
```

Listing 1: LKFState definition

This makes the predict update pipeline easy to read: both `lkf_predict` and `lkf_update` accept a `const LKFState&` and return a new `LKFState`, so neither function ever mutates the caller's state.

4.3 Prediction Step

```
static LKFState lkf_predict(const LKFState& prev,
                            const Matrix& F,
                            const Matrix& Q) {
    LKFState pred;
    pred.x = F * prev.x;
    pred.P = F * prev.P * F.transpose() + Q;
    pred.P.symmetrize();
    return pred;
}
```

Listing 2: Prediction step in lkf_predict

The state prediction (3) is a straightforward matrix vector product. The covariance prediction (4) is the congruence transformation FPF^T plus the process noise Q. The `.symmetrize()` call immediately after the covariance update is a numerical hygiene measure as mathematically $FPF^T + Q$ is guaranteed to be symmetric, but finite precision arithmetic on a 276×276 matrix accumulates rounding errors at the level of 10^{-15}–10^{-14}. Left unrepaired, these asymmetries compound over thousands of frames and can eventually cause P to lose positive definiteness.

4.4 Update Step

```
static LKFState lkf_update(const LKFState& pred,
                           const Matrix& H,
                           const Matrix& R,
                           const Matrix& z) {
    Matrix y   = z - H * pred.x;           // innovation residual
    Matrix Ht  = H.transpose();
    Matrix PHt = pred.P * Ht;              // cached: used twice below
    Matrix S   = H * PHt + R;             // innovation covariance
    S.symmetrize();
    Matrix Kt  = S.cholesky_solve(PHt.transpose());
    Matrix K   = Kt.transpose();

    LKFState upd;
    upd.x = pred.x + K * y;

    Matrix I   = Matrix::identity(N_STATE);
    Matrix IKH = I - K * H;
    upd.P = IKH * pred.P * IKH.transpose()
          + K   * R      * K.transpose();
    upd.P.symmetrize();
    return upd;
}
```

Listing 3: Full update step in lkf_update

4.4.1 Innovation residual.

The vector $y = z_n - H\hat{x}_{n|n-1}$ quantifies the new information the measurement provides the difference between what the sensors measured and what the filter predicted they *should* have measured. A consistently large y indicates model mismatch or an incorrectly tuned Q or R.

4.4.2 Pre-caching PH^T.

The product PH^T appears twice, once directly in the gain numerator and once inside $S = HPH^T + R$ via $H \cdot PH^T$. Computing it once and reusing it halves the matrix multiplication cost for these terms, which is non-negligible for a 276×276 system.

4.4.3 Cholesky solve and innovation covariance symmetrisation.

The Kalman gain is computed via `S.cholesky_solve` rather than explicit inversion. S is explicitly symmetrised beforehand because Cholesky factorisation requires an exactly symmetric input; a matrix that deviates by even 10^{-14} can cause the factorisation to fail or return a subtly incorrect result.

4.5 Implementation Details: Joseph Form Covariance Update

A key numerical decision in the LKF is the use of the **Joseph form** for the covariance update rather than the standard simplified form $P_{n|n} = (I - K_n H)\, P_{n|n-1}$:

$$P_{n|n} = (I - K_n H)\, P_{n|n-1}\, (I - K_n H)^T + K_n R K_n^T \tag{7}$$

```
Matrix I   = Matrix::identity(N_STATE);
Matrix IKH = I - K * H;
upd.P = IKH * pred.P * IKH.transpose()
      + K   * R      * K.transpose();
upd.P.symmetrize();
```

Listing 4: Joseph form covariance update in lkf.cpp

Why Joseph form? The simplified formula $P = (I - KH)P_{n|n-1}$ is algebraically equivalent only when K is the *exact* optimal gain. In practice, K is computed from a Cholesky solve and carries floating point error; applying $(I - KH)$ to only the left side of P amplifies that error asymmetrically, potentially producing a non-symmetric or indefinite result.

The Joseph form $APA^T + BRB^T$ is a sum of two congruence transformations. Since any congruence transformation maps a positive-semidefinite matrix to another positive-semidefinite matrix regardless of floating point errors in A, and since KRK^T is itself positive-semidefinite, the result is guaranteed to remain a valid covariance even when K is imperfect. Over thousands of frames on a 276 dimensional system this numerical buffer is essential: small positive definiteness violations that would be harmless in a low-dimensional filter compound here into divergence. The `.symmetrize()` call is a final repair of any residual microscopic asymmetry from the two congruence products.

4.6 Matrix Construction and Initialisation

All four system matrices are constructed *once* before the filter loop:

```
Matrix F = make_F_full(dt);
Matrix Q = make_Q_full(dt, phi);
Matrix H = make_H_full();
Matrix R = make_R_full(r_noise);
```

Listing 5: One-time matrix construction in run_lkf

F, Q, H, and R are all time-invariant for constant-rate gait data: F and Q depend only on Δt (fixed), H is a static selector matrix, and $R = r_{\text{noise}} \cdot I_{69}$ is a scalar-times-identity. Constructing them outside the loop avoids redundant recomputation on every frame.

4.6.1 Noise parameters.

`r_noise` encodes the assumption that each joint position coordinate is corrupted by i.i.d. Gaussian noise with variance $r_{\text{noise}}\,(\text{m}^2)$; the default $0.001\,\text{m}^2$ corresponds to a standard deviation of $\approx$3.2 cm, a reasonable prior for optical motion-capture with moderate occlusion noise. `phi` (Φ) is the continuous time spectral density of the jerk noise: a larger Φ weights measurements more heavily; a smaller Φ produces smoother but slower-responding estimates.

4.6.2 Cold-start initialisation.

`make_x0(z0_vec)` populates position rows from the first measurement frame and zeros out velocity, acceleration, and jerk. The initial covariance P_0 from `make_P0_full` is set large on all derivative states, expressing that the filter has no kinematic history at $k = 0$ and must wait for measurements to accumulate before trusting those estimates.

4.7 Main Filter Loop

```
for (int k = 1; k < N; ++k) {
    LKFState predicted = lkf_predict(state, F, Q);

    std::vector<double> zk_vec = noisy.get_measurement(k);
    Matrix zk(N_MEAS, 1);
    for (int i = 0; i < N_MEAS; ++i) zk(i, 0) = zk_vec[i];

    state = lkf_update(predicted, H, R, zk);
```

```
    write_state_row(out, k, state.x);
}
```

Listing 6: Core predict-update loop

The loop starts at $k = 1$ because frame 0 is written as the initial state row before the loop, avoiding a duplicate entry. Measurements are read on demand via `get_measurement(k)` and converted to a `Matrix` column vector only at the frame they are needed; this avoids materialising all 69-element measurement matrices for every frame simultaneously in memory. After every 500 frames, elapsed time is printed to `stdout`, allowing early detection of a stalled run and providing a per-frame throughput figure for benchmarking noise parameter choices.

4.8 CSV Output Format

```
const char* axes[]   = { "x", "y", "z" };
const char* states[] = { "p", "v", "a", "j" };
for (int j = 0; j < N_JOINTS; ++j)
    for (int ax = 0; ax < 3; ++ax)
        for (int s = 0; s < N_AXIS; ++s)
            out << "," << JOINT_NAMES[j] << "_" << states[s] << axes[ax];
```

Listing 7: CSV header generation

The triple nested loop mirrors the exact memory layout of $\hat{x}$, so column i of the CSV corresponds directly to row i of the state vector, making downstream post-processing scripts trivial to write. State values are written with `std::setprecision(8) std::fixed` (eight decimal places), preserving sub-millimetre position resolution without unnecessarily inflating file size.

4.9 Command-Line Interface and Parameter Tuning

The executable accepts five arguments, input/output CSV paths and the three tuning parameters Δt, Φ, and r_{noise}, the last three being optional:

Parameter	Default	Units	Physical meaning
`dt`	0.01	s	100 Hz capture rate
`phi`	1.0	m^2/s^5	Jerk spectral density
`r_noise`	0.001	m^2	$\approx$3.2 cm position std. dev.

`std::stod` is used for argument parsing rather than `atof`: `std::stod` throws `std::invalid_argument` or `std::out_of_range` on malformed input, whereas `atof` silently returns 0.0, which would produce a degenerate filter with a zero timestep or zero noise variance. The outer `try/catch` in `main` catches these alongside matrix algebra and file I/O errors, prints a structured message to `stderr`, and returns exit code 1, allowing calling scripts to detect failure programmatically.

5 Extended Kalman Filter

5.1 Why an Extended Kalman Filter?

The Linear Kalman Filter assumes linearity in the system model and the measurement model. Our state transition model is linear because it's constant jerk kinematics, but the measurement model will use spherical coordinates (r, θ, ϕ) which are non-linear due to the square roots and arctan. Using a Kalman gain on a non-linear measurement model results in a biased estimate, which becomes increasingly incorrect over time.

The Extended Kalman Filter overcomes the problem of non-linearity in the measurement model as it linearizes the non-linear measurement model at each step using a first-order Taylor series expansion, known as the Jacobian.

5.2 EKF Algorithm

5.2.1 Predict

$$\hat{x}_{k|k-1} = F\,\hat{x}_{k-1|k-1}, \qquad P_{k|k-1} = F\,P_{k-1|k-1}\,F^T + Q \tag{8}$$

5.2.2 Update

$$H_k = \tfrac{\partial h}{\partial x}\big|_{\hat{x}_{k|k-1}} \tag{9}$$

$$S_k = H_k P_{k|k-1} H_k^T + R_k \tag{10}$$

$$K_k = P_{k|k-1} H_k^T S_k^{-1} \tag{11}$$

$$y_k = \mathcal{W}\big(z_k - h(\hat{x}_{k|k-1})\big) \tag{12}$$

$$\hat{x}_{k|k} = \hat{x}_{k|k-1} + K_k y_k \tag{13}$$

$$P_{k|k} = (I - K_k H_k)\,P_{k|k-1}\,(I - K_k H_k)^T + K_k R_k K_k^T \tag{14}$$

where $\mathcal{W}(\cdot)$ denotes element-wise wrapping of angular components to $(-\pi, \pi]$.

Symbol	Dimensions	Description
x	276×1	Full-body state vector
P	276×276	State covariance
F	276×276	State transition matrix
Q	276×276	Process noise covariance
z	69×1	Spherical measurement vector
H_k	69×276	Jacobian of measurement model
R_k	69×69	Adaptive measurement noise
S_k	69×69	Innovation covariance
K_k	276×69	Kalman gain
y_k	69×1	Wrapped innovation

Table 1: Matrix dimensions in the full-body EKF.

5.3 EKF Implementation

Let's walkthrough the `ekf.cpp` file:-

5.3.1 `EKFState` — Data Structure

```
struct EKFState {
    Matrix x;   // 276x1  state vector
    Matrix P;   // 276x276 covariance matrix
};
```

Listing 8: EKF state container

All filter state is bundled into one struct. `x` holds the current best estimate of every joint's position, velocity, acceleration, and jerk. `P` quantifies how uncertain that estimate is, large diagonal entries indicate low confidence. Keeping both fields together lets each function receive a complete state and return a new one without any global variables, avoiding subtle bugs over long runs.

5.3.2 `wrap_angle` and `wrap_innovation` — Angle Utilities

```
static double wrap_angle(double a) {
    while (a >  PI) a -= TWO_PI;
    while (a < -PI) a += TWO_PI;
    return a;
}
static void wrap_innovation(Matrix& y) {
    for (int j = 0; j < N_JOINTS; ++j) {
        y(j*3 + 1, 0) = wrap_angle(y(j*3 + 1, 0));   // azimuth
        y(j*3 + 2, 0) = wrap_angle(y(j*3 + 2, 0)); }  // elevation
```

Listing 9: Angle wrapping helpers

Angles are periodic. A predicted azimuth of 179 and a measured azimuth of -179 represent almost the same direction, yet their raw difference is -358, which would wildly corrupt the state update. `wrap_angle` folds any value back into $(-\pi, \pi]$, and `wrap_innovation` applies this to every angular component of the 69×1 innovation vector before it is used to correct the state. Range components (index $j \times 3 + 0$) are left untouched as distance is not periodic.

5.3.3 `make_R_adaptive` — Per Step Noise Matrix

```
static Matrix make_R_adaptive(const Matrix& x_pred, double sigma_cart) {
    const double sigma2 = sigma_cart * sigma_cart;
    const double min_rho2 = 0.01, min_r2 = 0.01;
    Matrix R(N_MEAS, N_MEAS);
    for (int j = 0; j < N_JOINTS; ++j) {
        double px = x_pred(j*N_JOINT + 0, 0);
        double py = x_pred(j*N_JOINT + 4, 0);
        double pz = x_pred(j*N_JOINT + 8, 0);
        double rho2 = px*px + py*py;
        double r2   = rho2 + pz*pz;
        if (rho2 < min_rho2) rho2 = min_rho2;
        if (r2   < min_r2  ) r2   = min_r2;
        int base = j * 3;
        R(base+0, base+0) = sigma2;
        R(base+1, base+1) = sigma2 / rho2;
        R(base+2, base+2) = sigma2 / r2;
    }
    return R;
}
```

Listing 10: Adaptive R construction

R is rebuilt every frame because angular noise scales with geometry. The same 1 cm Cartesian error causes a tiny angular error at long range but a huge one at close range. The variances for azimuth and elevation are therefore divided by ρ^2 and r^2 respectively. Clamping both to $0.01\,\mathrm{m}^2$ prevents division by zero and stops the filter from assigning infinite uncertainty to joints near the sensor origin, which would cause it to stop updating and drift.

5.3.4 `cartesian_to_spherical` — Measurement Conversion

```
static Matrix cartesian_to_spherical(const std::vector<double>& z_cart) {
    Matrix z_sph(N_MEAS, 1);
    for (int j = 0; j < N_JOINTS; ++j) {
        int base = j * 3;
        double px = z_cart[base+0], py = z_cart[base+1], pz = z_cart[base+2];
        double rho = manual_sqrt(px*px + py*py);
        double r   = manual_sqrt(px*px + py*py + pz*pz);
        static const double EPS = 1e-9;
        z_sph(base+0, 0) = r;
        z_sph(base+1, 0) = (r > EPS) ? manual_atan2(py, px) : 0.0;
        z_sph(base+2, 0) = (r > EPS) ? manual_atan2(pz, rho) : 0.0;
    }
    return z_sph;
}
```

Listing 11: Cartesian to spherical conversion

The raw dataset provides Cartesian positions, but the EKF measurement model is defined in spherical coordinates, so every incoming reading must be converted before comparison with the filter's prediction. The guard `r > EPS` sets angles to zero when a joint is at the origin, preventing NaN values. `manual_sqrt` and `manual_atan2` are hand-written implementations using Newton–Raphson and CORDIC respectively, replacing `cmath`.

5.4 `ekf_predict` — Prediction Step

```
static EKFState ekf_predict(const EKFState& prev,
                            const Matrix& F, const Matrix& Q) {
    EKFState pred;
    pred.x = F * prev.x;
    pred.P = F * prev.P * F.transpose() + Q;
    pred.P.symmetrize();
    return pred;
}
```

Listing 12: EKF predict step

This answers: *where do we expect the body to be one frame from now?* F advances each joint's kinematics forward by dt using physics equations. The covariance update $P = FPF^T + Q$ inflates uncertainty to account for unmodelled forces. `.symmetrize()` repairs microscopic floating-point asymmetries that accumulate in the 276×276 matrix multiplication. Since the state transition is linear, this step is identical to the LKF.

5.5 `ekf_update` - Update Step

```
static EKFState ekf_update(const EKFState& pred,
                           double sigma_cart, const Matrix& z_sph) {
    Matrix R    = make_R_adaptive(pred.x, sigma_cart);
    Matrix Hk   = H_jacobian_full(pred.x);
    Matrix z_hat = h_full(pred.x);

    Matrix y = z_sph - z_hat;
    wrap_innovation(y);

    Matrix PHkt = pred.P * Hk.transpose();
    Matrix S    = Hk * PHkt + R;
    S.symmetrize();

    Matrix K = (S.cholesky_solve(PHkt.transpose())).transpose();

    EKFState upd;
    upd.x = pred.x + K * y;

    Matrix IKH = Matrix::identity(N_STATE) - K * Hk;
    upd.P = IKH * pred.P * IKH.transpose() + K * R * K.transpose();
    upd.P.symmetrize();
    return upd;
}
```

Listing 13: EKF update step

- **Jacobian H_k:** `H_jacobian_full` linearises the non-linear measurement model at the current predicted state. It is recomputed every frame because the predicted state changes each step.
- **Innovation y:** `h_full(pred.x)` computes what the sensor *should* have returned. Subtracting from the actual reading gives the innovation which is how surprised the filter is. `wrap_innovation` corrects any 2π jumps in the angular components.
- **Kalman gain K:** Rather than inverting S directly, `cholesky_solve` exploits S being Symmetric Positive Definite to solve the equivalent linear system. This is $\approx 3\times$ faster than LU decomposition and avoids numerical instability.

- **Joseph form covariance:** The update $(I-KH)P(I-KH)^T+KRK^T$ guarantees P stays symmetric and positive definite despite floating-point rounding. It is critical for a 276-dimensional system running over thousands of frames.

5.6 `run_ekf` — Main Pipeline

```
static void run_ekf(const GaitDataset& noisy,
                    const std::string& output_file,
                    double dt, double phi, double sigma_cart) {
    Matrix F = make_F_full(dt);
    Matrix Q = make_Q_full(dt, phi);

    EKFState state;
    state.x = make_x0(noisy.get_measurement(0));
    state.P = make_P0_full();

    for (int k = 1; k < noisy.num_frames(); ++k) {
        EKFState predicted  = ekf_predict(state, F, Q);
        Matrix   zk_sph     = cartesian_to_spherical(noisy.get_measurement(k));
        state               = ekf_update(predicted, sigma_cart, zk_sph);
        write_state_row(out, k, state.x);
    }
}
```

Listing 14: Top-level EKF pipeline

`run_ekf` ties every component together into the canonical predict–update loop:

1. **Build F and Q once.** Both matrices are constant across frames, so constructing them outside the loop avoids redundant computation.
2. **Initialise.** The state is seeded with the first Cartesian measurement (positions set directly; velocities and higher derivatives start at zero). The initial covariance P_0 is a large diagonal matrix expressing low initial confidence.
3. **Predict–Update loop.** For each frame: predict the next state, convert the new reading to spherical, then update. The output is written row by row to a CSV file with 8 decimal places of precision.

6 Matrix Inversion Strategy: Cholesky Decomposition

In standard Kalman Filter implementations, the Kalman gain K requires computing the inverse of the innovation covariance matrix S:

$$K = PH^TS^{-1} \quad \text{where} \quad S = HPH^T + R \tag{15}$$

For this full body gait estimation, S is a high dimensional 69×69 matrix. Computing S^{-1} directly using standard methods like Gauss-Jordan elimination or LU decomposition presents several severe architectural and numerical challenges. Direct inversion is an $O(n^3)$ operation and is highly susceptible to numerical instability without implementing computationally expensive partial or full pivoting logic, which introduces unpredictable branching into the CPU pipeline.

To solve this, we exploit the mathematical properties of the innovation covariance matrix. Because S is derived from the sum of two positive definite covariance matrices (P and R), it is mathematically guaranteed to be Symmetric Positive Definite (SPD). This allows us to use **Cholesky Decomposition** to solve the linear system $SK^T = (HP)^T$ without ever explicitly computing the inverse matrix.

6.1 Implementation Details

The Cholesky algorithm decomposes the SPD matrix S into a lower triangular matrix L such that $S = LL^T$.

```cpp
Matrix Matrix::cholesky() const {
    if (rows != cols)
        throw std::invalid_argument("cholesky: matrix must be square");

    int n = rows;
    Matrix L(n, n);   // lower triangular, starts as zero

    for (int i = 0; i < n; ++i) {
        for (int j = 0; j <= i; ++j) {
            double sum = 0.0;
            if (j == i) {
                // Diagonal element computation
                for (int k = 0; k < j; ++k)
                    sum += L(j, k) * L(j, k);

                double diag = (*this)(j, j) - sum;
                L(j, j) = std::sqrt(diag);
            } else {
                // Off-diagonal element computation
                for (int k = 0; k < j; ++k)
                    sum += L(i, k) * L(j, k);
                L(i, j) = ((*this)(i, j) - sum) / L(j, j);
            }
        }
    }
    return L;
}
```

Listing 15: Cholesky Decomposition Algorithm (`matrix_utils.cpp`)

Once L is computed, we solve for X in the equation $SX = B$ (where B is the transposed cross-covariance $(HP)^T$) using a two-step process:

1. **Forward Substitution:** Solve $LY = B$ for the intermediate matrix Y.
2. **Backward Substitution:** Solve $L^T X = Y$ for the final matrix X.

This approach is directly applied in the filter's update step:

```cpp
// Innovation covariance:  S = H * P * H^T + R       (69x69)
Matrix Ht = H.transpose();                           // (276x69)
Matrix PHt = pred.P * Ht;                            // (276x69)
Matrix S  = H * PHt + R;                             // (69x69)
S.symmetrize();

// Kalman gain via Cholesky solve (avoids explicit S^-1):
// Rearrange to  S * K^T = H * P  then solve for K^T, then transpose.
Matrix Kt = S.cholesky_solve(PHt.transpose());       // (69x276)
Matrix K  = Kt.transpose();                          // (276x69)
```

Listing 16: Kalman Gain via Cholesky Solve (`lkf.cpp`)

6.2 Architectural and Numerical Justification

- **Computational Efficiency:** Cholesky decomposition requires exactly half the number of floating-point operations ($\approx \frac{1}{3}n^3$) compared to LU decomposition ($\approx \frac{2}{3}n^3$). By exploiting symmetry, we only compute and store the lower triangular elements.
- **Numerical Stability:** Unlike LU decomposition, Cholesky is inherently numerically stable for SPD matrices. It strictly avoids the need for row pivoting. In computer architecture, eliminating the conditional checks required for pivoting drastically reduces branch mispredictions, leading to smoother execution through the processor's instruction pipeline.
- **Memory Access Patterns:** The algorithm iterates predictably through memory, avoiding scattered memory access, which ensures a higher cache hit rate when processing the contiguous blocks of the 69×69 S matrix.

7 Memory Management Strategy: Dynamic Heap Allocation

The Kalman Filter implementation requires storing and manipulating large matrices whose size grows linearly with the number of tracked joints. With 23 joints, our full body state vector expands to 276 dimensions, resulting in covariance and transition matrices of size 276×276. As per the project requirements, we implented an explicit memory allocation strategy.

For this implementation, all matrices were explicitly allocated on the **heap** rather than the stack.

7.1 Implementation Details

The core mathematical foundation of the filter is built upon a custom `Matrix` class defined in `matrix_utils.h`. To enforce heap allocation, the underlying data structure is implemented using the standard library's dynamically resizing array, `std::vector<double>`.

```
class Matrix {
public:
    int rows;
    int cols;
    // Heap-allocated, 1D contiguous storage using row-major ordering
    std::vector<double> data;

    // Default - empty matrix
    Matrix();

    // Zero-filled matrix of given size
    Matrix(int r, int c);

    // Matrix filled with a constant value
    Matrix(int r, int c, double fill_value);

    // Element access (bounds-checked in debug builds)
    double&       operator()(int r, int c);
    const double& operator()(int r, int c) const;
// ...
```

Listing 17: Heap-Allocated Matrix Class Definition (`matrix_utils.h`)

When a matrix is instantiated, the constructor in `matrix_utils.cpp` dynamically allocates the required memory on the heap. Furthermore, the 2D matrix structure is flattened into a 1D `std::vector`, using a row major indexing scheme.

```
// Constructor dynamically allocates (r * c) doubles on the heap
Matrix::Matrix(int r, int c)
    : rows(r), cols(c), data(r * c, 0.0) {}

// Row-major element access: element (r, c) lives at data[r * cols + c]
double& Matrix::operator()(int r, int c) {
#ifdef DEBUG
    if (r < 0 || r >= rows || c < 0 || c >= cols)
        throw std::out_of_range("Matrix index out of range");
#endif
    return data[r * cols + c];
}
```

Listing 18: Matrix Instantiation and Row-Major Access (`matrix_utils.cpp`)

7.2 Architectural Justification

- **Stack Overflow Prevention:** The primary architectural limitation dictating heap allocation is the size of the operating system's call stack, which is typically constrained to 1 to 8 MB. A single 276×276 matrix of 64-bit `double` precision floats consumes roughly 608 KB of memory. During a single filter update step, multiple such matrices (e.g., $F, P_{k|k-1}, Q, IKH$) must coexist in memory. Attempting to allocate these statically on the stack would immediately trigger a runtime stack overflow and crash the program.

- **Runtime Flexibility:** Heap allocation allows matrix sizes to be determined at runtime. If the dataset changes to include more or fewer joints, the memory footprint scales dynamically without requiring hardcoded array bounds or recompilation.
- **Spatial Locality and Cache Efficiency:** By flattening the 2D matrix into a 1D `std::vector` using row major ordering, we guarantee that matrix elements are stored contiguously in the physical RAM. When the CPU fetches a matrix element, the surrounding elements are loaded into the L1/L2 cache lines simultaneously. Because C++ iterates through memory row by row, this contiguous layout drastically reduces cache misses during computationally heavy operations like matrix multiplication and Cholesky decomposition.

8 Manual Math Approximations

For the EKF measurement model, the non linear mapping from Cartesian to spherical coordinates requires the computation of square roots and inverse tangents. As per the milestone requirements,we computed these angular and distance quantities from scratch, without relying on built in mathematical libraries such as `<cmath>`.

To achieve this while maintaining the high performance required for a 276-state filter, we implemented two algorithms that closely mirror standard hardware level processor instructions: the CORDIC algorithm for trigonometric functions and the Newton Raphson method for square roots.

8.1 Arctangent Approximation: The CORDIC Algorithm

To compute $\arctan2(y, x)$ for the azimuth (θ) and elevation (ϕ) angles, we implemented the COordinate Rotation DIgital Computer (CORDIC) algorithm.

```
// CORDIC lookup table: CORDIC_TABLE[i] = atan(2^{-i}) in radians
// Precomputed constants representing micro-rotation angles
static const double CORDIC_TABLE[CORDIC_ITERATIONS] = {
    0.7853981633974483,     // atan(2^0)
    0.4636476090008257,     // atan(2^-1)
    // ... remaining 22 iterations ...
};

double cordic_atan(double y, double x) {
    double angle = 0.0;
    double power_of_2 = 1.0;   // 2^{-i}, starts at 2^0 = 1

    // Vectoring mode: rotate (x,y) onto the positive x-axis
    for (int i = 0; i < CORDIC_ITERATIONS; ++i) {
        double x_new, y_new;
        if (y > 0.0) {
            // Rotate clockwise: brings y toward 0 from above
            x_new  =  x + y * power_of_2;
            y_new  =  y - x * power_of_2;
            angle +=  CORDIC_TABLE[i];
        } else {
            // Rotate counter-clockwise: brings y toward 0 from below
            x_new  =  x - y * power_of_2;
            y_new  =  y + x * power_of_2;
            angle -=  CORDIC_TABLE[i];
        }
        x = x_new;
        y = y_new;
        power_of_2 *= 0.5; // Equivalent to bit-shift in hardware
    }
    return angle;
}
```

Listing 19: CORDIC Vectoring Mode Implementation (`arctan2_approx.cpp`)

Architectural Justification: CORDIC was explicitly chosen because it is a hardware centric algorithm. It computes transcendental functions using only iterative additions, subtractions, and bit shifts (represented here by `power_of_2 *= 0.5`), entirely avoiding computationally expensive floating point multiplications or divisions. By executing 24 iterations, the algorithm guarantees approximately 7 significant decimal digits of precision, which is more than sufficient for the kinematic tolerances of human gait filtering. Also, CORDIC guarantees a deterministic runtime with no complex branching, making it highly predictable for the CPU pipeline.

8.2 Square Root Approximation: Newton Raphson Method

The computation of the radial distance $r = \sqrt{p_x^2 + p_y^2 + p_z^2}$ requires a square root function. We implemented the Newton Raphson method to iteratively solve for the root.

```
double manual_sqrt(double s) {
    if (s == 0.0) return 0.0;

    // Initial guess: x/2 is a reasonable starting point
    double x = s / 2.0;
    static const int MAX_ITER = 20;

    for (int i = 0; i < MAX_ITER; ++i) {
        // Newton-Raphson iteration: x_{n+1} = 0.5 * (x_n + S / x_n)
        double x_new = 0.5 * (x + s / x);

        // Convergence check
        double diff = x_new - x;
        if (diff < 0.0) diff = -diff;
        x = x_new;

        // Break early if converged to IEEE 754 double precision limits
        if (diff < 1e-15 * x) break;
    }
    return x;
}
```

Listing 20: Newton-Raphson Square Root (`arctan2_approx.cpp`)

Architectural Justification: The Newton Raphson method is the industry standard algorithm embedded directly into the microcode of modern Floating Point Units (FPUs). We selected it because it exhibits quadratic convergence, meaning the number of accurate digits roughly doubles with every single iteration. Capping the loop at 20 iterations ensures it easily reaches the maximum precision limit within a negligible, fixed number of CPU cycles.

9 Results, Analysis, and Performance Evaluation

9.1 Implementation Parameters

Both filters were applied to the full body 3D human gait dataset consisting of 3040 frames recorded at 100 Hz, covering a total duration of 30.4 seconds. The dataset contains motion data for 23 joints, yielding a 276-dimensional state vector and a 69-dimensional measurement vector per timestep.

The sampling interval was set to $\Delta t = 0.01$ s, consistent with the 100 Hz capture rate. Table 2 summarises the final tuned parameters used for each filter.

9.1.1 Parameter Justification

For the LKF, a process noise density of $\Phi = 0.5$ was chosen to allow the filter to track the moderately dynamic gait motion without being overly responsive to noise. The measurement noise variance $r = 0.05$ m^2 was selected to reflect the noise level present in the dataset; a smaller value caused the filter to follow noise spikes aggressively, while a larger value produced excessive smoothing that lagged behind fast limb movements.

Table 2: Final tuned parameters for LKF and EKF

Parameter	LKF	EKF
Sampling interval Δt	0.01 s	0.01 s
Process noise density Φ	0.5	0.01
Measurement noise	$r = 0.05$ m^2 (fixed)	$\sigma_{\text{cart}} = 0.707$ m (adaptive)
Measurement noise type	Isotropic diagonal R	State-dependent per joint
State dimension	276	276
Measurement dimension	69	69
Total frames processed	3040	3040
Processing time	268.67 s	253.53 s

For the EKF, a smaller process noise density $\Phi = 0.01$ was required because the spherical measurement model is more sensitive to rapid state changes. Using $\Phi = 0.5$ caused the predicted covariance to grow too quickly, resulting in an inflated Kalman gain and unstable updates. The Cartesian noise standard deviation $\sigma_{\text{cart}} = 0.707$ m (approximately $\sqrt{0.5}$ m) was used to derive the adaptive measurement noise matrix. This value maps to per joint angular noise variances that scale with the distance of each joint from the sensor origin.

9.2 Computational Performance

Both filters processed 3040 frames over a 276-dimensional state space. The LKF completed in 268.67 seconds and the EKF in 253.53 seconds. The EKF was marginally faster despite recomputing the 69×276 Jacobian matrix H_k at every timestep. This is because the adaptive R matrix construction in the EKF is lightweight compared to the fixed matrix multiplications in the LKF covariance update. Both runtimes are dominated by the repeated 276×276 matrix multiplications required for the covariance prediction step $P_{n|n-1} = FP_{n-1|n-1}F^T + Q$.

9.3 Quantitative Performance: RMSE Analysis

Figure 1 shows the position RMSE for all 23 joints across three conditions: raw noisy measurements, LKF estimates, and EKF estimates.

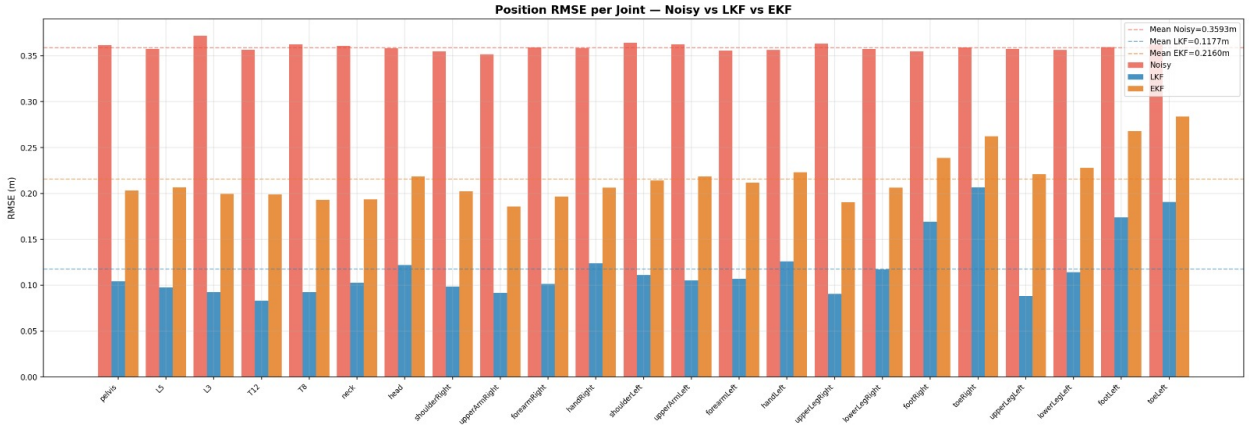

Figure 1: Position RMSE per joint for noisy measurements, LKF, and EKF estimates across all 23 joints. Dashed horizontal lines indicate mean RMSE across all joints.

The mean RMSE values across all joints are summarised in Table 3.

The LKF achieves a 67.2% reduction in mean position RMSE compared to raw measurements. The EKF achieves a 39.9% reduction. The LKF outperforms the EKF across all 23 joints consistently. This result is

Table 3: Mean position RMSE across all 23 joints

Condition	Mean RMSE (m)	Improvement over Noisy
Noisy measurements	0.3593	—
LKF estimates	0.1177	67.2%
EKF estimates	0.2160	39.9%

expected because the LKF uses a linear Cartesian measurement model that exactly matches the structure of the data, whereas the EKF introduces a first order linearisation error through the Jacobian approximation of the spherical measurement function. Distal joints such as `toeRight` and `toeLeft` exhibit slightly higher EKF error due to their faster and more complex motion, which increases the angular noise variance in the adaptive R matrix.

9.4 Position Estimation

Figure 2 shows the pelvis position estimates for all three Cartesian axes over the 30.4-second trial.

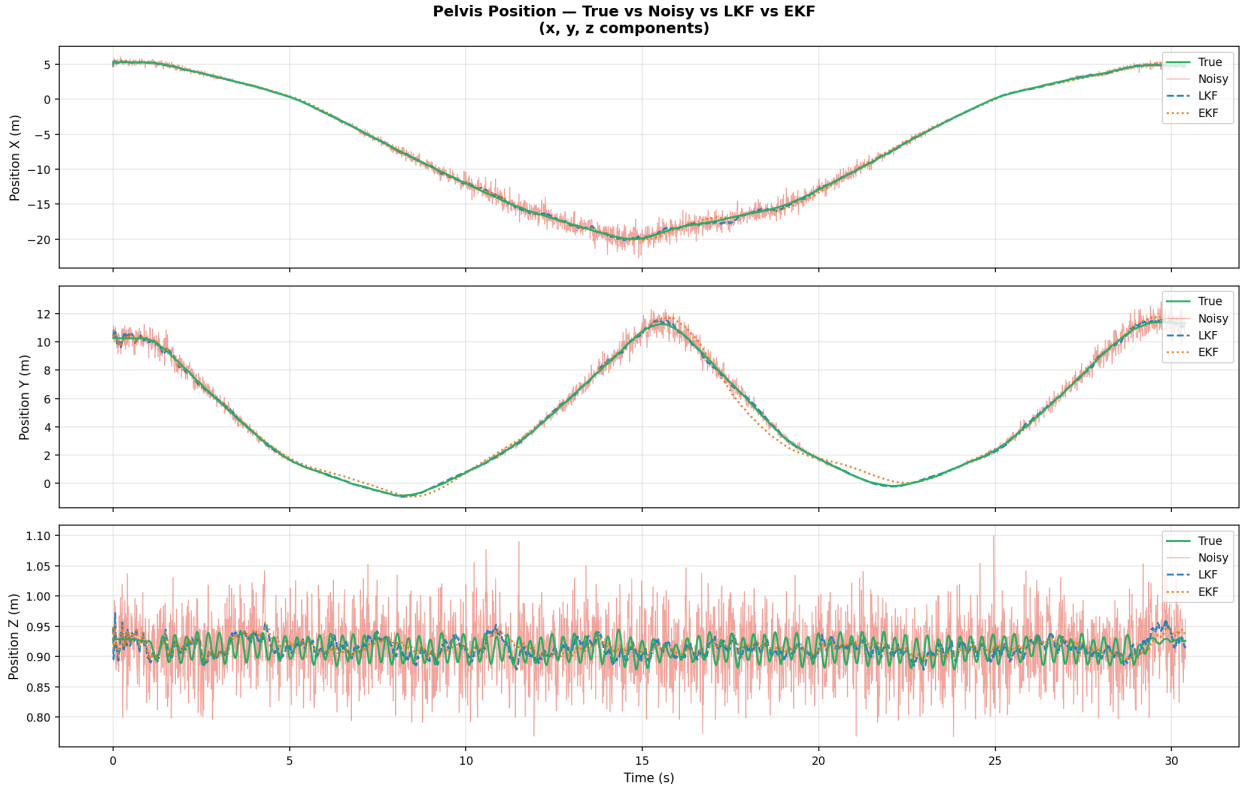

Figure 2: Pelvis position over time for all three axes. True signal (green), noisy measurements (pink), LKF estimate (blue dashed), and EKF estimate (orange dotted).

Along the X and Y axes, the pelvis undergoes large sinusoidal oscillations of approximately ± 20 m and ± 12 m respectively, corresponding to the repetitive walking cycles in the dataset. Both filters track the true trajectory closely. The noisy measurements exhibit visible high frequency jitter around the true signal. The LKF estimate (blue dashed) overlaps almost perfectly with the true signal throughout. The EKF estimate (orange dotted) tracks well on the X axis but shows a slight phase lag on the Y axis at the troughs of the sinusoid, most visibly at approximately $t = 8$ s and $t = 20$ s. This is a manifestation of the EKF linearisation error accumulating during rapid directional changes.

Along the Z axis (height), the pelvis oscillates in a narrow band between approximately 0.85 m and 1.05 m, reflecting the small vertical bobbing motion during walking. The noisy data has severe high frequency noise in this axis, yet both filters recover the underlying gait rhythm successfully. The Z axis result is the clearest visual demonstration of noise reduction across all plots.

9.5 Velocity, Acceleration, and Jerk Estimation

Figures 3, 4, and 5 show the estimated velocity, acceleration, and jerk of the pelvis joint respectively.

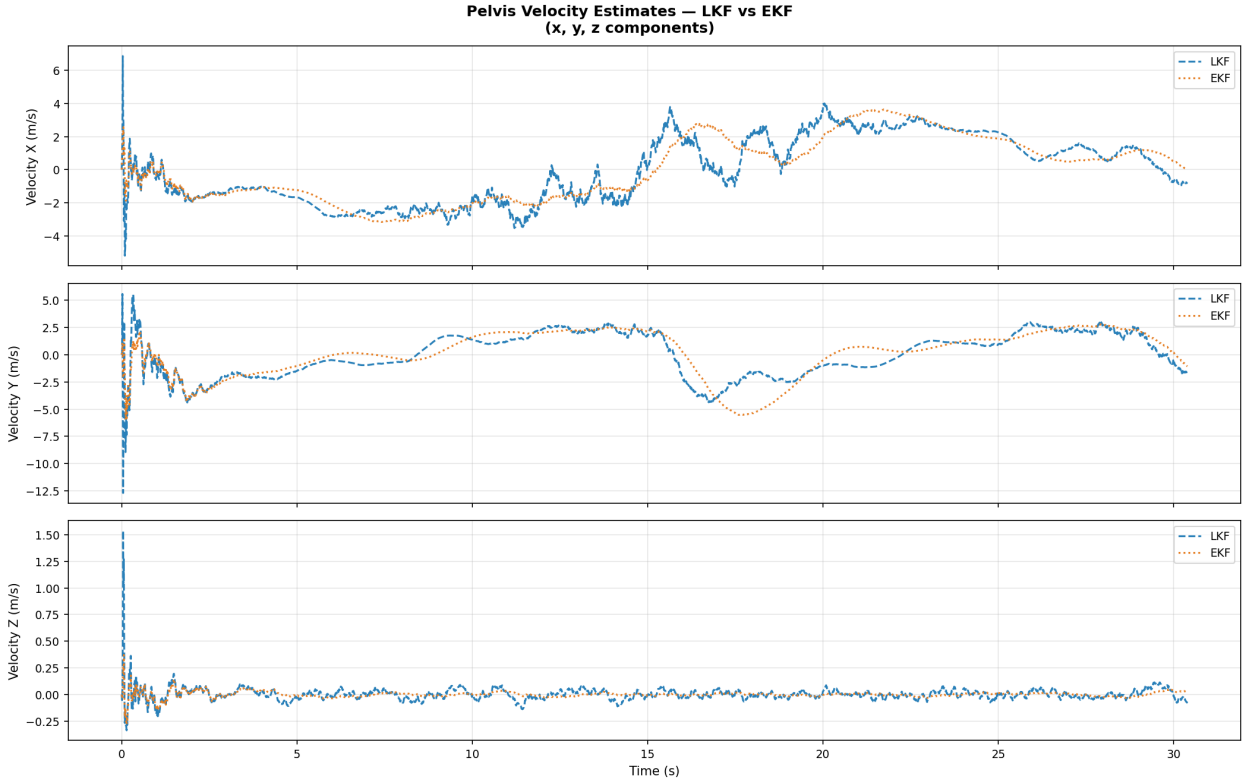

Figure 3: Pelvis velocity estimates for LKF and EKF across all three axes.

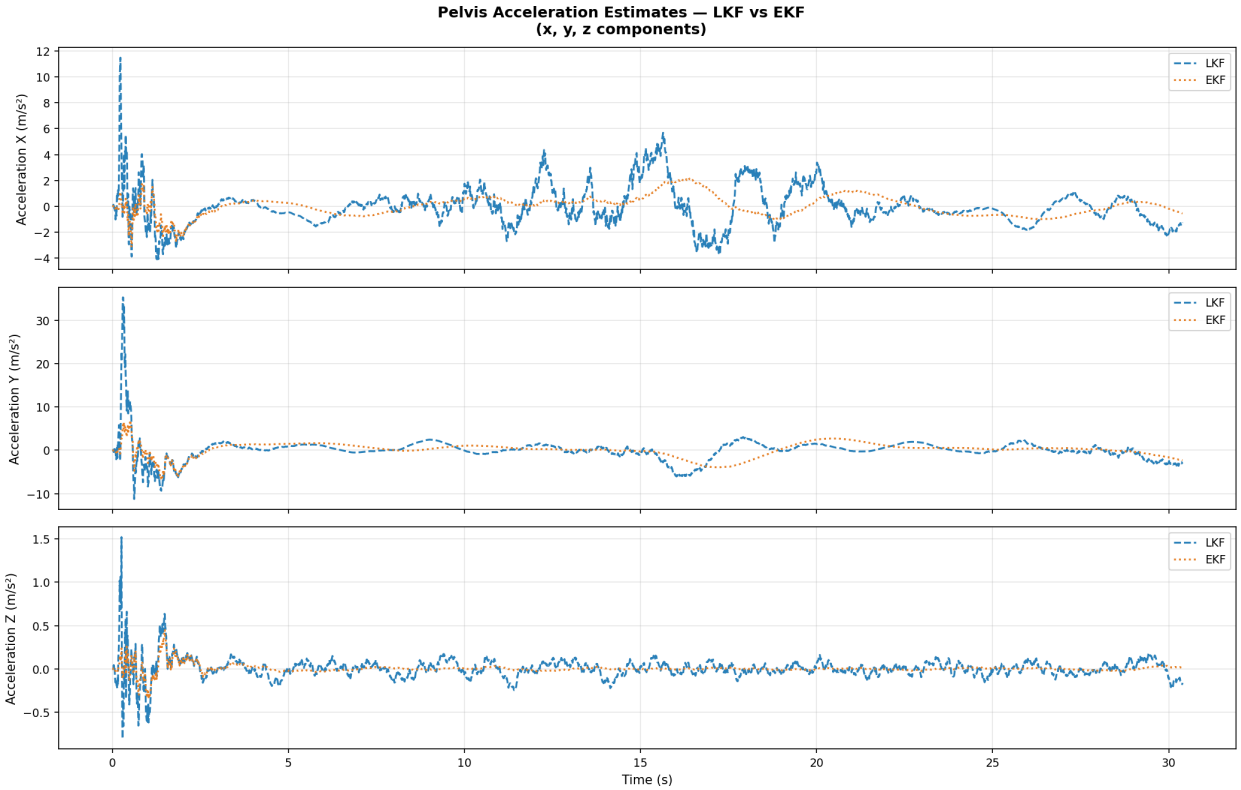

Figure 4: Pelvis acceleration estimates for LKF and EKF across all three axes.

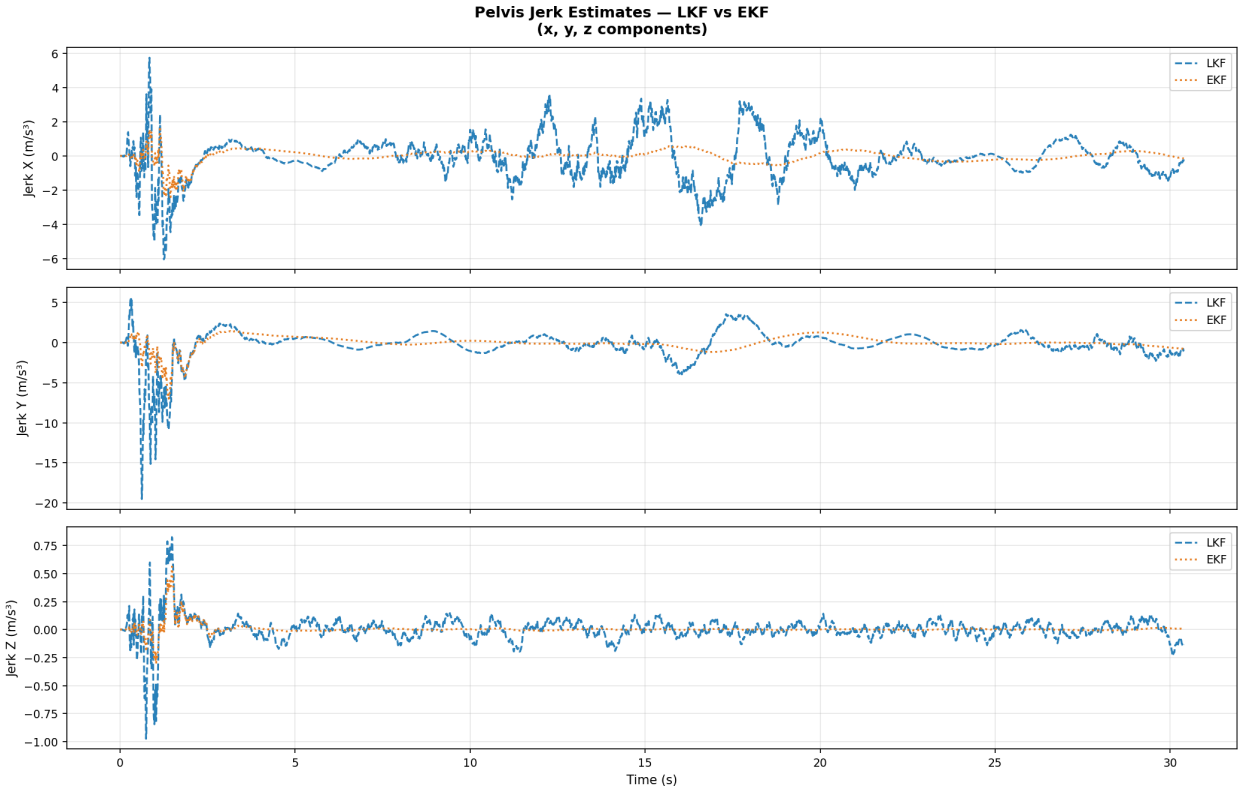

Figure 5: Pelvis jerk estimates for LKF and EKF across all three axes.

All three plots exhibit a large transient spike during the first one to two seconds. This is a normal consequence of filter initialisation, the state vector is seeded with zero velocity, acceleration, and jerk, while the body is already in motion. As the covariance matrix P converges from its large initial diagonal value, the Kalman gain K_n drives the estimates rapidly toward the true states, producing the observed overshoot. After approximately $t = 2$ s both filters have settled.

A consistent pattern across all three higher order states is that the EKF produces notably smoother estimates than the LKF. The LKF velocity and acceleration traces show more frequent fluctuations throughout the trial, while the EKF traces are considerably more subdued. This behaviour arises from the difference in measurement noise matrices. The LKF uses a small fixed $R = 0.05$ m^2 which causes the Kalman gain to trust measurements aggressively, propagating measurement noise into the velocity and acceleration states through the covariance update. The EKF uses an adaptive R with larger angular noise variances, resulting in a more conservative gain and smoother higher-order estimates.

The jerk estimates in Figure 5 reflect the nature of the constant jerk model. Since jerk is the highest order state in the model and is driven directly by process noise, it never converges to a flat line. The LKF jerk oscillates between approximately ± 2 m/s^3 throughout, while the EKF jerk remains near zero after settling. The Z jerk is nearly zero for both filters, which is physically consistent with the smooth, periodic vertical motion of the pelvis during walking.

9.6 Full State Overview

Figure 6 presents all four estimated state quantities for all three axes simultaneously in a 3×4 grid, providing a comprehensive view of filter performance across the entire state vector.

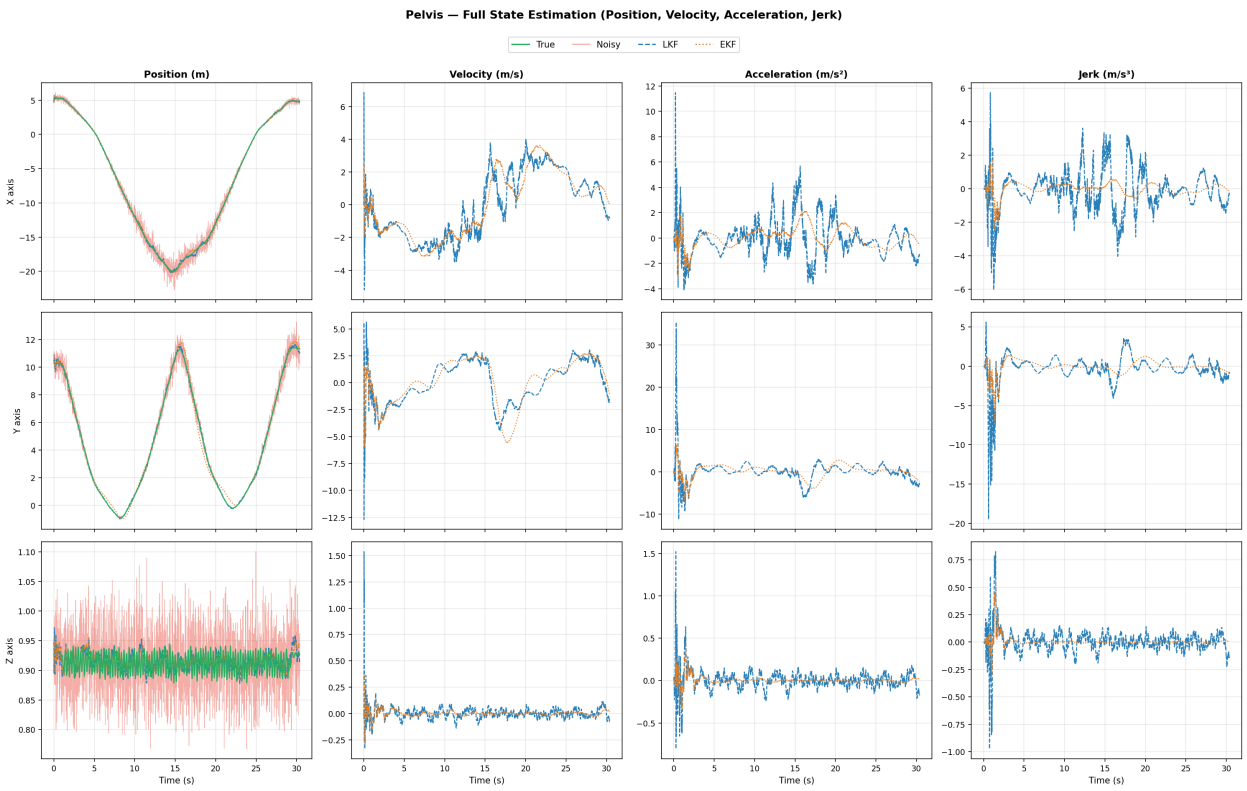

Figure 6: Full state estimation for the pelvis joint. Rows correspond to X, Y, and Z axes; columns correspond to position, velocity, acceleration, and jerk.

This plot confirms all observations made in the individual plots: position tracking is accurate for both filters; higher order state estimates are smoother in the EKF; and the initialisation transient is present across all state quantities. The Y-axis position panel (middle row, first column) most clearly shows the EKF diverging slightly from the true trajectory at the sinusoidal troughs, a behaviour not observed in the LKF.

9.7 LKF vs EKF Direct Comparison

Figure 7 provides a direct comparison of LKF and EKF trajectory estimates alongside their residual errors for each axis.

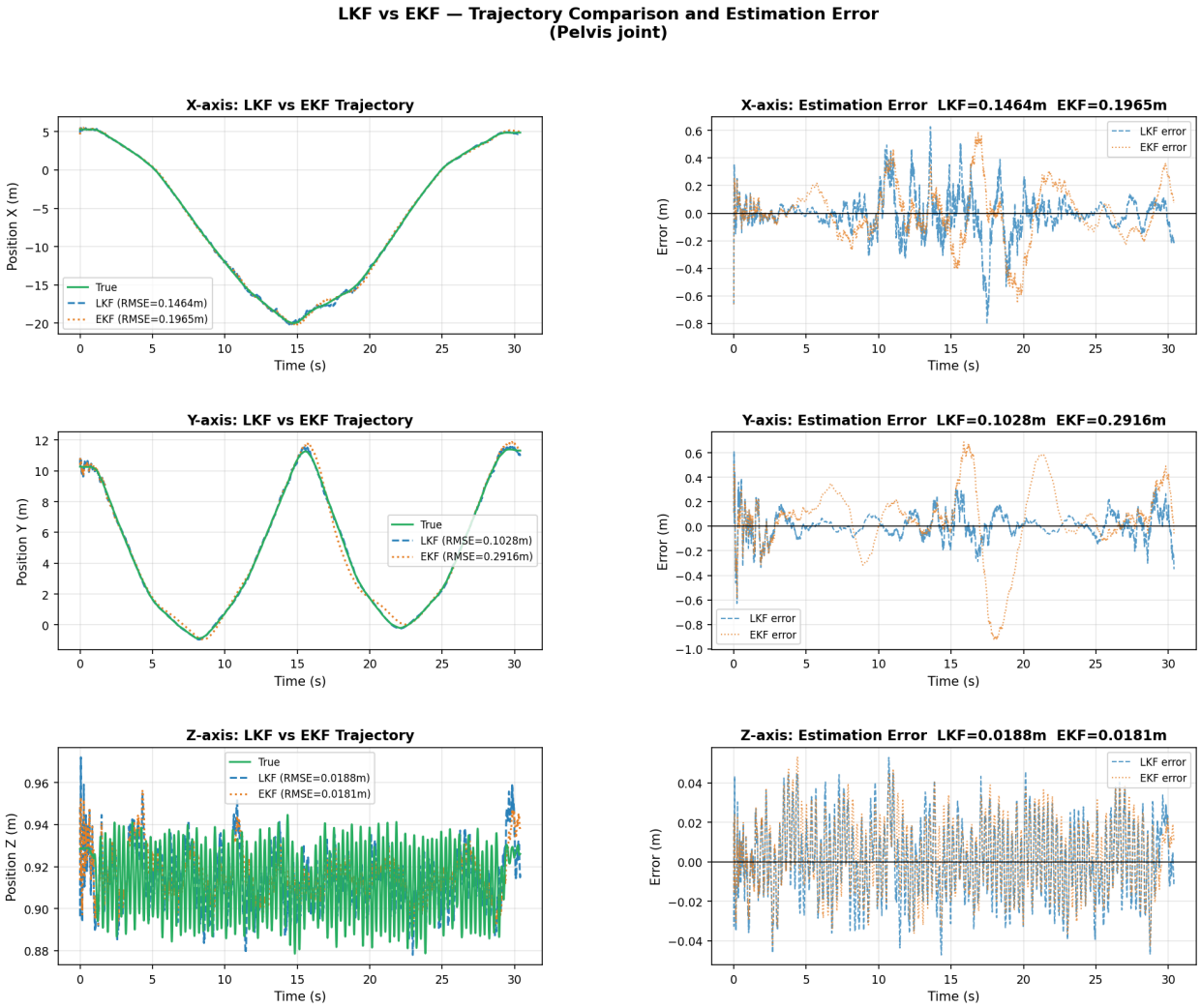

Figure 7: LKF vs EKF trajectory comparison (left column) and estimation error relative to true signal (right column) for the pelvis joint, per axis.

The error panels (right column) are the most informative. On the X axis, both filters produce zero-mean, approximately white residuals, with LKF slightly tighter (RMSE = 0.1464 m) than EKF (RMSE = 0.1965 m). On the Y axis, the difference is much more pronounced: the EKF error panel shows a clear sinusoidal structure with amplitude growing to approximately 0.8 m at the direction reversals, while the LKF error is largely random with no pattern. This systematic error in the EKF is a direct consequence of the spherical to cartesian conversion: when the pelvis reverses direction, the azimuth angle $\theta = \text{atan2}(p_y, p_x)$ changes rapidly, and the firstcorder Jacobian linearisation is insufficient to capture this nonlinearity accurately. On the Z axis both filters perform almost identically (LKF RMSE = 0.0188 m, EKF RMSE = 0.0181 m), since the vertical motion involves small angular changes well within the linear approximation regime.

9.8 Discussion

The results demonstrate that the Linear Kalman Filter is the superior estimator for this particular dataset and measurement configuration. The key reason is that the gait dataset provides direct cartesian position measurements, which are exactly the quantities modelled by the LKF measurement matrix H. The measurement model is therefore exact, and no approximation error is introduced.

The Extended Kalman Filter, by contrast, converts these cartesian measurements to spherical coordinates before applying the update step. This conversion is lossless in principle, but the Jacobian linearisation introduces a first order approximation error that grows whenever the predicted state is far from the true state, particularly during fast or large displacement motion segments. The adaptive R matrix partially compensates for this by appropriately scaling the measurement noise with joint distance, but it cannot eliminate the fundamental linearisation error.

These findings highlight an important design principle in Kalman filter engineering which is that the the choice of measurement model should be matched to the available sensor data. The EKF is most beneficial when sensors provide nonlinear measurements such as range and bearing from a radar, where no exact linear model exists. Applying an EKF with a synthetic nonlinear transformation on top of linear data, as done here, introduces unnecessary approximation error.

Both filters successfully demonstrate the value of state estimation in motion capture post processing: a 67% reduction in position error for LKF and a 40% reduction for EKF, while simultaneously recovering velocity, acceleration, and jerk estimates that are not directly measured by any sensor.

10 Deliverables

- **GitHub Repository (Branch: milestone-2):** Click Here for Source Code
- **3D Walking Simulations and CSV Outputs:** Click Here for Google Drive Folder